

DoDo Bot — Perception Stack

Gesture Recognition Model — MediaPipe + HaGRID

This report documents training a static hand-gesture classifier for DoDo Bot, including a MediaPipe API breaking change encountered mid-pipeline, a model comparison (Random Forest vs. neural network), and a data-driven decision to drop a confused gesture class.

Metric	Value
Dataset	HaGRID (hagrid-classification-512p), 11.7GB
Feature extraction	MediaPipe HandLandmarker — 21 landmarks x (x,y,z) = 63 features
Detection yield	2,777 / 3,000 images (92.5%)
Final gesture classes	9 (palm, fist, ok, peace, like, dislike, call, one, mute)
Final model	Random Forest, 200 trees
Final accuracy	0.94 (up from 0.89 with 10 classes)

Objective

Train a lightweight classifier that recognizes static hand gestures from images, using MediaPipe hand landmarks as input features rather than raw pixels — small enough to eventually run in real time on the robot's onboard camera. This replaces an earlier emotion-detection track, since gestures are more directly actionable for robot behavior.

Gesture Set — Adjusted from Original Plan

The original Day 3 plan listed gestures including wave, beckoning, both-hands-up, and swipe. HaGRID is a static, single-hand dataset, so gestures requiring motion or two hands aren't available in it. The gesture set was mapped to HaGRID's actual classes instead:

HaGRID Class	Gesture	Robot Response
palm	Open palm	Halt movement
fist	Fist	Emergency stop
ok	OK sign	Task-complete confirmation
peace	Peace / V	Menu / bill request
like	Thumbs up	Order confirmed
dislike	Thumbs down	Order cancelled / issue flagged
call	Call gesture	Alert staff (custom mapping)
one	Index pointing	Directional command

mute	Mute/quiet	Do-not-disturb request
------	------------	------------------------

Note: **two_up** was initially included as a 10th class (mapped to a custom "navigation command") but was dropped after diagnosis below.

Step-by-Step Walkthrough

1. Downloaded HaGRID via kagglehub

The full classification-512p subset (11.7GB) downloaded and extracted cleanly, no authentication issues encountered.

2. Hit a MediaPipe breaking-change error

The original plan's landmark extraction code used ``mp.solutions.hands``, which failed with:

```
AttributeError: module 'mediapipe' has no attribute 'solutions'
```

3. Diagnosed and fixed via the Tasks API

Checking the installed version showed MediaPipe 1.0.0 — confirming the legacy "solutions" interface (deprecated since 2023) had been fully removed. Fixed by switching to the current ``HandLandmarker`` Tasks API, which requires downloading a ``.task`` model file and wrapping images in ``mp.Image`` objects before detection. Same 63-feature output (21 landmarks x x/y/z), different API surface.

4. Extracted landmarks from 10 gesture classes

300 images per class (3,000 total) processed; 2,777 (92.5%) yielded a detectable hand. The 7.5% failures are expected — occlusion, awkward crops, etc.

5. Trained and compared two classifiers

A Random Forest baseline (89% accuracy) and a small neural network (88% accuracy) were trained on identical data and evaluated the same way. Near-identical scores across two structurally different model types was itself informative for the next step.

6. Diagnosed a specific class confusion

The confusion matrix showed ``peace`` and ``two_up`` were heavily confused with each other in BOTH models (roughly 20-26 misclassifications each direction, out of ~55 samples per class). Since two different model architectures hit the identical wall on the identical class pair, the issue was concluded to be inherent to the landmark features for these two visually similar gestures — not fixable by swapping classifiers.

7. Made a reasoned decision to drop two_up

``peace`` maps to an intuitive, likely-to-be-used action (menu/bill request); ``two_up`` was an arbitrary custom mapping (navigation command) that ``one`` (pointing) can reasonably substitute for. Chose to drop ``two_up`` rather than merge the classes, preserving ``peace`` as a clean, distinct gesture.

8. Retrained on the final 9 classes

Accuracy rose from 89% to 94%; ``peace``'s f1-score alone rose from 0.66 to 0.98, confirming the diagnosis was correct.

Results — Before vs. After Dropping two_up

Before (10 classes, Random Forest):

Class	Precision	Recall	F1
peace	0.73	0.60	0.66
two_up	0.65	0.73	0.69
ALL (10 classes)	0.89	0.89	0.89

After (9 classes, two_up dropped, Random Forest):

Class	Precision	Recall	F1
peace	0.98	0.98	0.98
ALL (9 classes)	0.94	0.94	0.94

Known Limitation

A minor residual confusion remains between `like` and `call` (thumbs-up vs. the "call me" gesture — both involve an extended thumb), causing about 8 misclassifications in validation. This is a much smaller effect than the peace/two_up issue and was not addressed further tonight; worth monitoring if these two robot actions are safety- or order-critical.

Why Random Forest Over the Neural Network for Deployment

Both models scored within 1 point of each other (94% RF-equivalent vs. ~88-89% NN on 10 classes). Given near-identical accuracy, Random Forest was chosen for the final deployed model since it trains faster, needs no GPU for inference, and is simpler to run on the robot's onboard hardware later.

Next Steps

Add a baseline "no gesture / idle hand" class (not present in HaGRID — would need to be sourced separately, e.g. from frames with no detected hand). Integrate the classifier with a live camera feed once hardware is available, using the same MediaPipe HandLandmarker pipeline on video frames instead of static images. Separately: implement the ROS2 distance/safety-zone node.